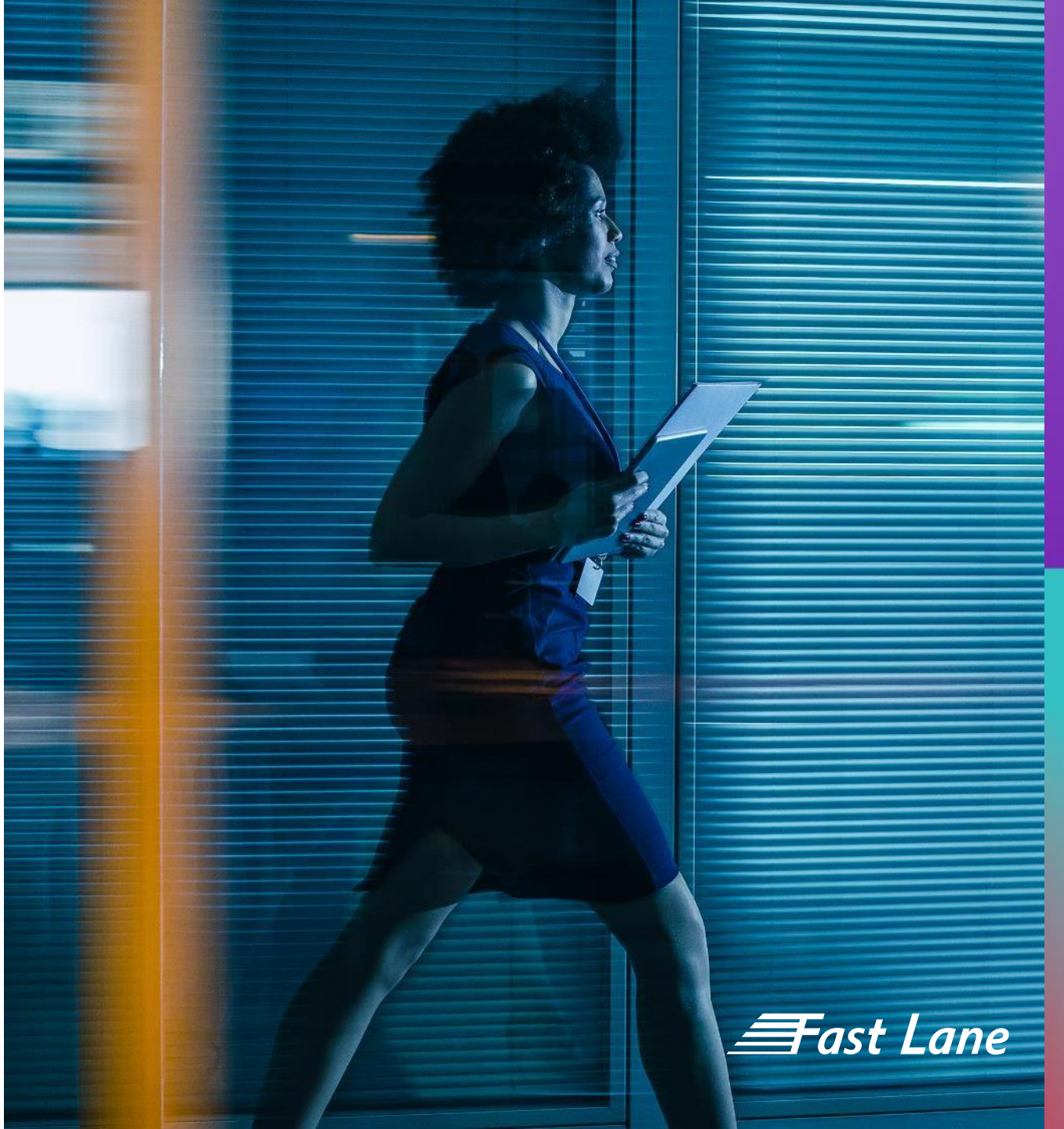
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 7

Dapr Building Blocks for Portable Services

The Contoso Logistics flow on sidecars and component files

Event-Driven Distributed Apps on Azure: Aspire, Dapr & Durable Execution



01

Dapr Sidecar: Run with Aspire

dapr init, AddDapr, WithDaprSidecar and component YAML

02

Service Invocation: mTLS & Retries

App IDs, InvokeMethodAsync and Sentry certificates

03

State Stores: ETags & Concurrency

ETag checks, transactions and the Cosmos DB store

04

Pub/Sub: Swap Brokers via Components

The Topic attribute, dead-letter topics and Service Bus

05

Resiliency Policies: Retries & Breakers

Timeouts, retries and circuit breakers in one spec

06

Dapr Actors: Virtual Actor Model

VanActor, turn-based calls, timers and reminders

07

Managed Dapr: Container Apps vs. AKS

Component tiers, platform limits and the version window

Dapr building blocks for portable services

The Contoso Logistics flow on sidecars and component files

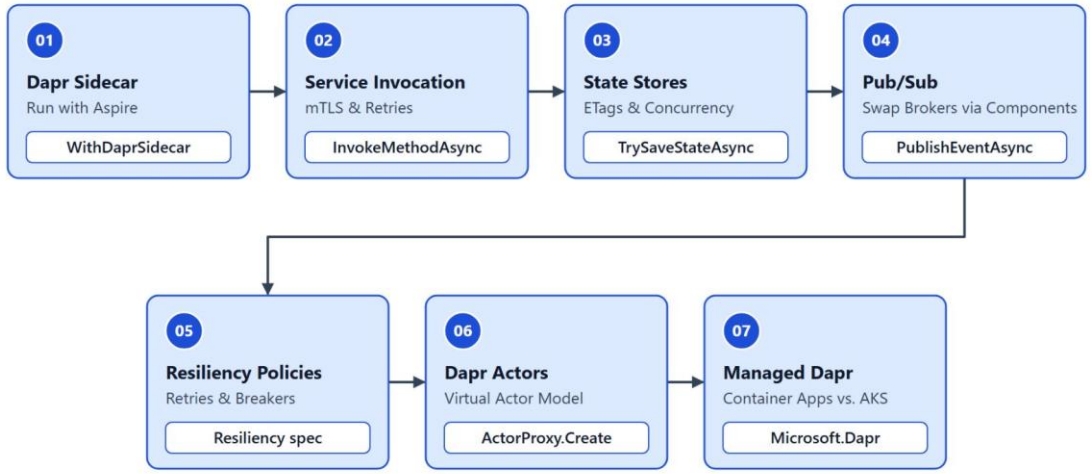
- **Sidecar and service calls**: Dapr beside each service in the Aspire AppHost, calls by app ID
- **State and events**: ETag checks on writes, pub/sub through a broker set in a component file
- **Policies, actors and hosting**: retries and breakers, one `VanActor` per van, Container Apps or AKS



Overview

The path through the seven topics

- Start the sidecar from the Aspire AppHost and load the component files
- Call `routing-service` by app ID, then write state with **ETag** checks
- Publish scan events through a broker you can swap in YAML
- Put timeouts, retries and breakers in one **resiliency spec**, model each van as an actor
- Decide per service: Container Apps or AKS



01

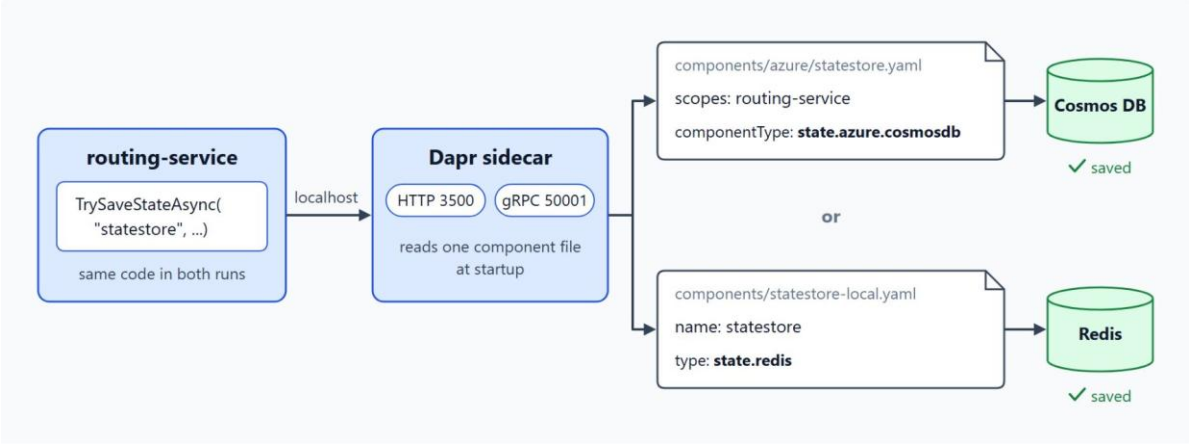
Dapr Sidecar: Run with Aspire

dapr init, AddDapr, WithDaprSidecar and component YAML

The sidecar model

The service calls localhost and the component file picks the store

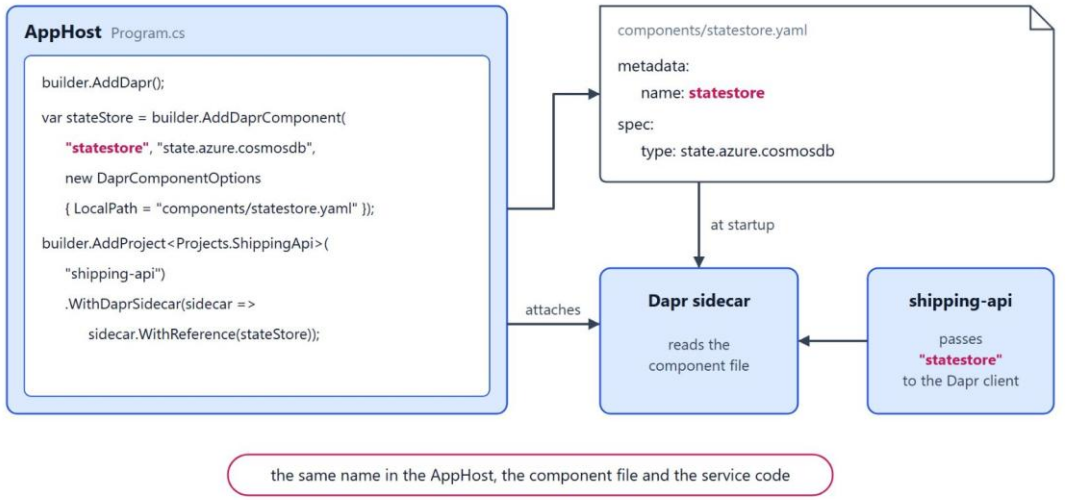
- Dapr runs as a separate process next to each service
- The service calls its sidecar on localhost: HTTP on port 3500, gRPC on 50001
- Building blocks: service invocation, state, pub/sub, bindings, actors, secrets, configuration
- Each block delegates to a **component**: Cosmos DB, Service Bus or Redis
- `TrySaveStateAsync("statestore", ...)` stays the same in both runs



Dapr in the Aspire AppHost

Install the CLI and attach a sidecar to each service

- `winget install Dapr.CLI`, then `dapr init`, then check `dapr --version`
- `dapr init` starts Redis, Zipkin, placement and scheduler; `--slim` skips them
- `AddDapr`, `AddDaprComponent` and `WithDaprSidecar` come from `CommunityToolkit.Aspire.Hosting.Dapr`
- Use one component name in the AppHost, the YAML `metadata.name` and the service code
- This class targets Dapr 1.18: CLI and runtime both report 1.18.0



02

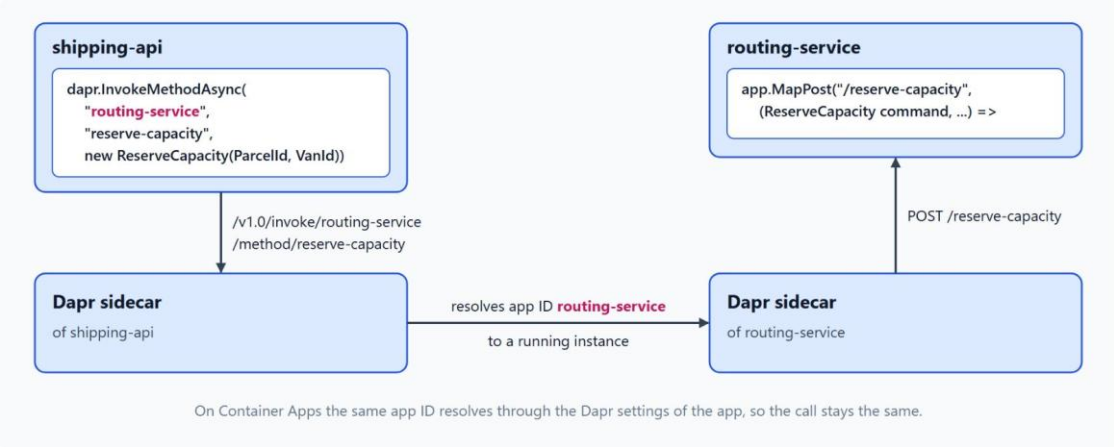
Service Invocation: mTLS & Retries

App IDs, InvokeMethodAsync and Sentry certificates

Calling a service by app ID

The sidecar resolves routing-service to a running instance

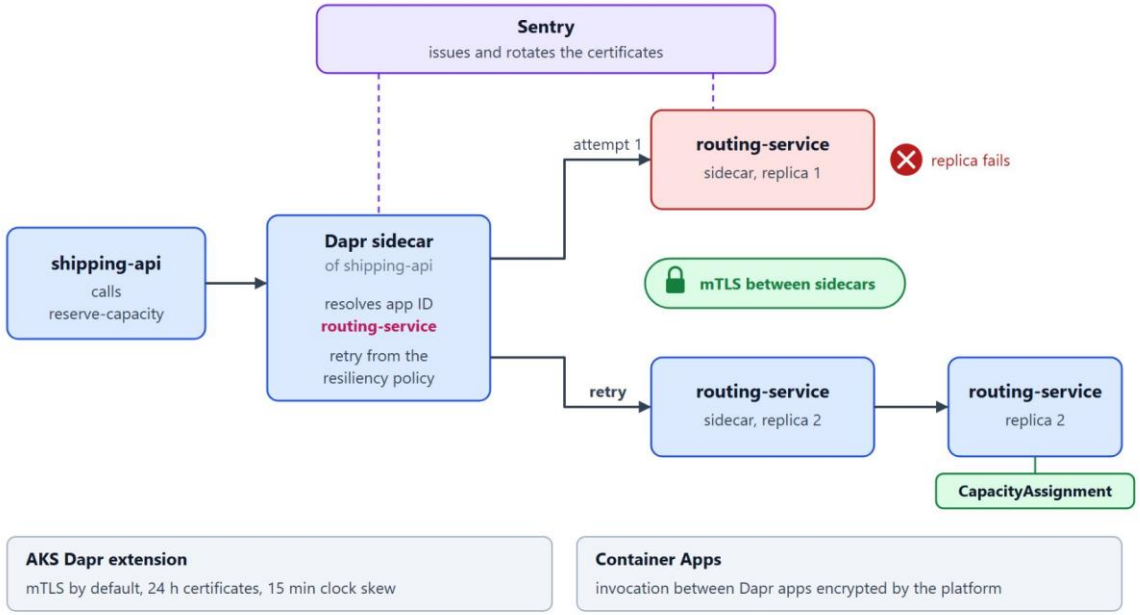
- Every Dapr app has an **app ID**, shared by all of its replicas
- Callers use `/v1.0/invoke/routing-service/method/reserve-capacity` on the sidecar
- The sidecar resolves the ID through Kubernetes DNS, or mDNS when self-hosted
- `InvokeMethodAsync` handles serialization, `CreateInvokeHttpClient` returns an `HttpClient`
- The `HttpRequestMessage` overloads are obsolete in the current SDK



Mutual TLS and retries

Sentry issues the certificates, the resiliency policy decides the retries

- **Sentry** issues and rotates the certificates; application code never handles them
- The AKS extension turns mTLS on by default: 24 h certificates, 15 min clock skew
- Container Apps encrypts invocation between Dapr-enabled apps through the platform
- Retries with backoff come from the resiliency policy for `routing-service`
- When the caller needs no answer, publish an event instead



03

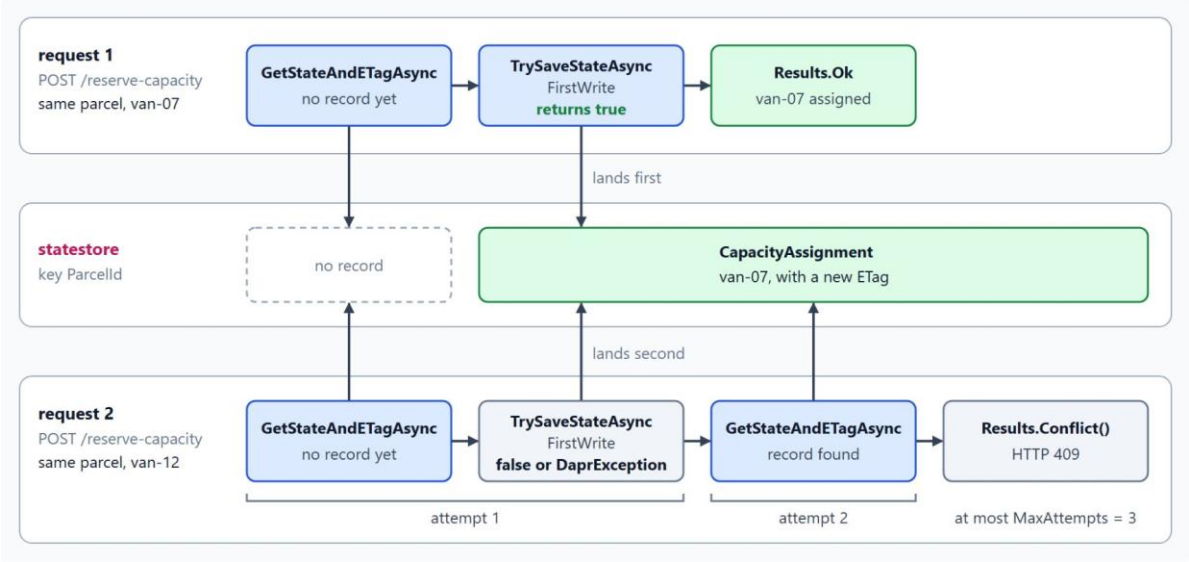
State Stores: ETags & Concurrency

ETag checks, transactions and the Cosmos DB store

ETags and first-write-wins

A write succeeds only while the ETag still matches

- `GetStateAndETagAsync` returns the value together with its **ETag**
- `TrySaveStateAsync` with `ConcurrencyMode.FirstWrite` returns `false` when another writer won
- The Redis state store throws a `DaprException` for the same conflict
- On a conflict, read again and recompute in a short loop
- After a fixed number of attempts, return a conflict to the caller



Write modes and transactions

Choosing a concurrency mode and changing several keys at once

- Without an ETag the newest write replaces the value: **last-write-wins**
- Use first-write-wins for a transition that must happen once
- `StateOptions.Consistency`: eventual by default, strong waits for replicas or a quorum
- `ExecuteStateTransactionAsync` succeeds for every key or fails for every key
- `state.azure.cosmosdb` is Tier 1 and needs the partition key path `/partitionKey`



04

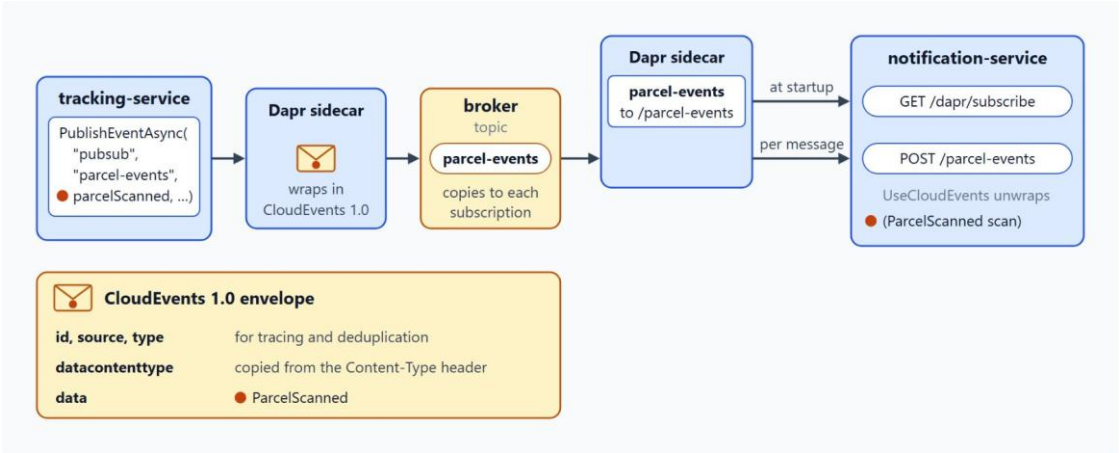
Pub/Sub: Swap Brokers via Components

The Topic attribute, dead-letter topics and Service Bus

Publishing and subscribing

The code names the component pubsub and the topic parcel-events

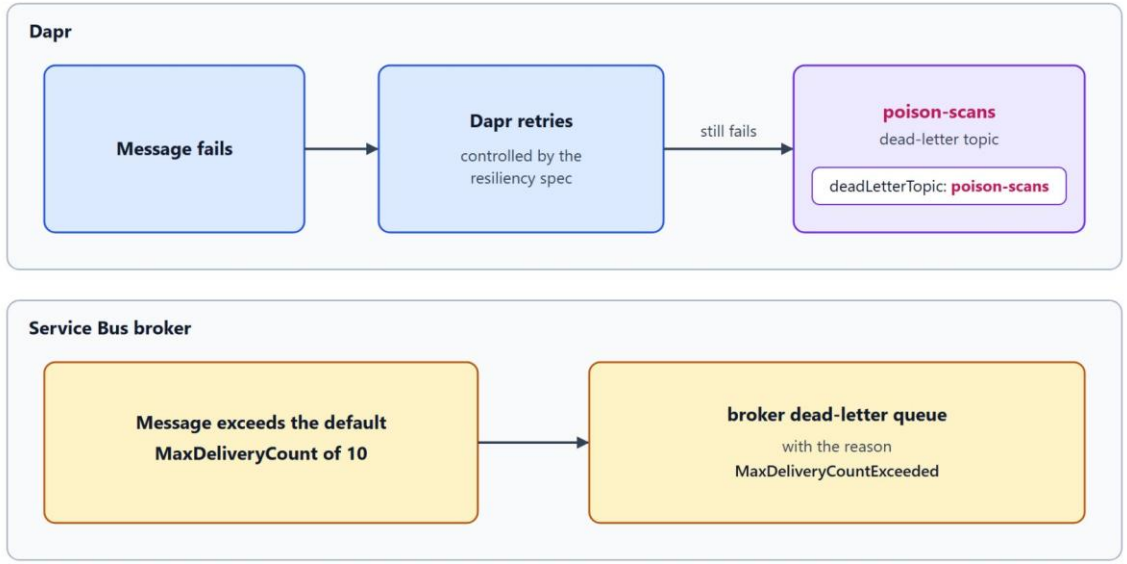
- `PublishEventAsync("pubsub", "parcel-events", scan)` names no broker
- `[Topic("pubsub", "parcel-events")]` or `WithTopic` declares the subscription
- `MapSubscribeHandler` lists the subscriptions, `UseCloudEvents` unwraps the **CloudEvents** envelope
- Swap the broker in the component file: Service Bus, Event Hubs, Redis or Kafka
- Use the native Service Bus SDK when the logic depends on session state



Dead-letter topics

Messages that still fail after the retries land on poison-scans

- Declare a subscription in a `dapr.io/v2alpha1 Subscription` file or in code
- `deadLetterTopic: poison-scans` takes what still fails after the retries
- The resiliency spec controls how Dapr retries
- Service Bus dead-letters a message after the default `MaxDeliveryCount` of 10
- `maxDeliveryCount` takes effect only when Dapr creates the subscription



05

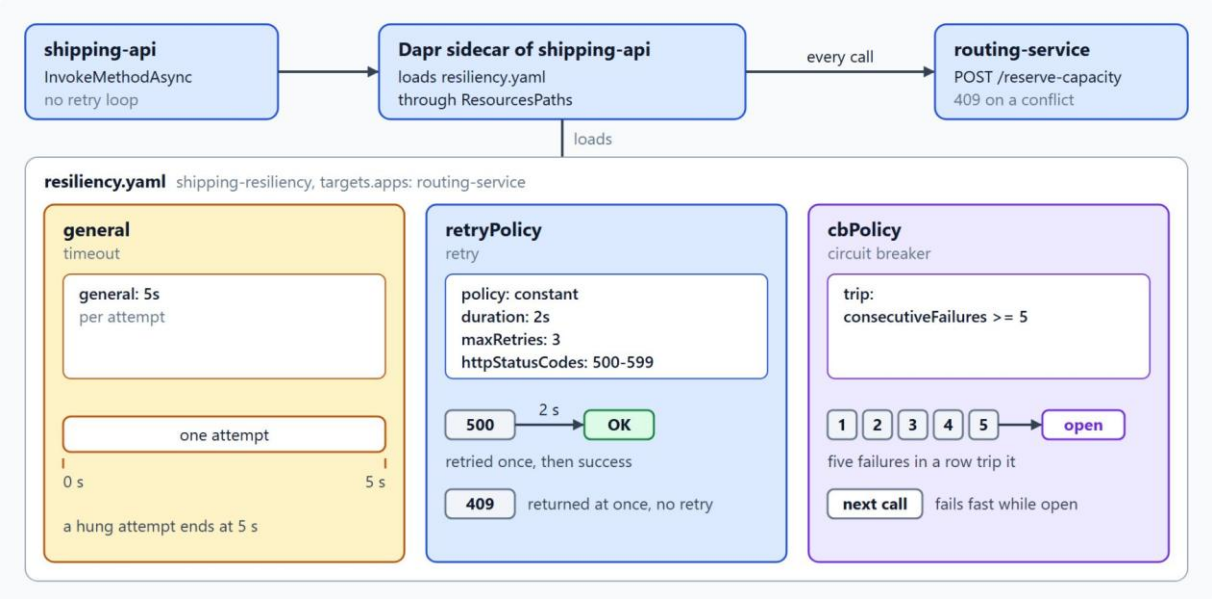
Resiliency Policies: Retries & Breakers

Timeouts, retries and circuit breakers in one spec

The resiliency spec

Named policies in one file, applied to calls to routing-service

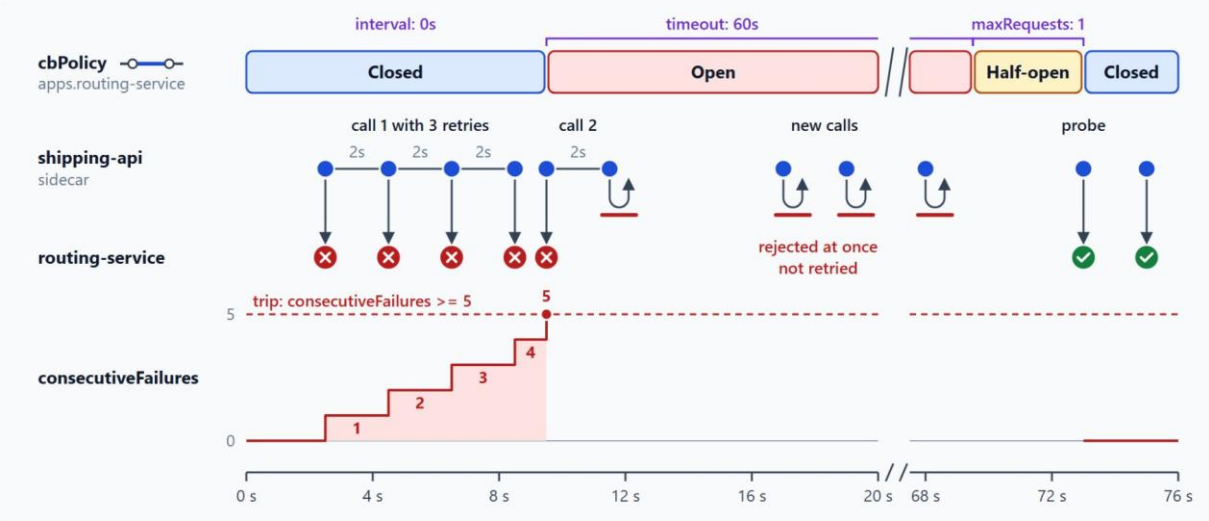
- `policies` names timeouts, retries and circuit breakers; `targets` attaches them
- `general`: each attempt waits at most 5 s
- `retryPolicy`: constant 2 s delay, 3 retries, only status codes 500 to 599
- A full call can take the timeout times the attempts, plus the delays
- The sidecar loads the file only when the AppHost sets `ResourcesPaths`



How the circuit breaker opens and closes

cbPolicy trips after five consecutive failures

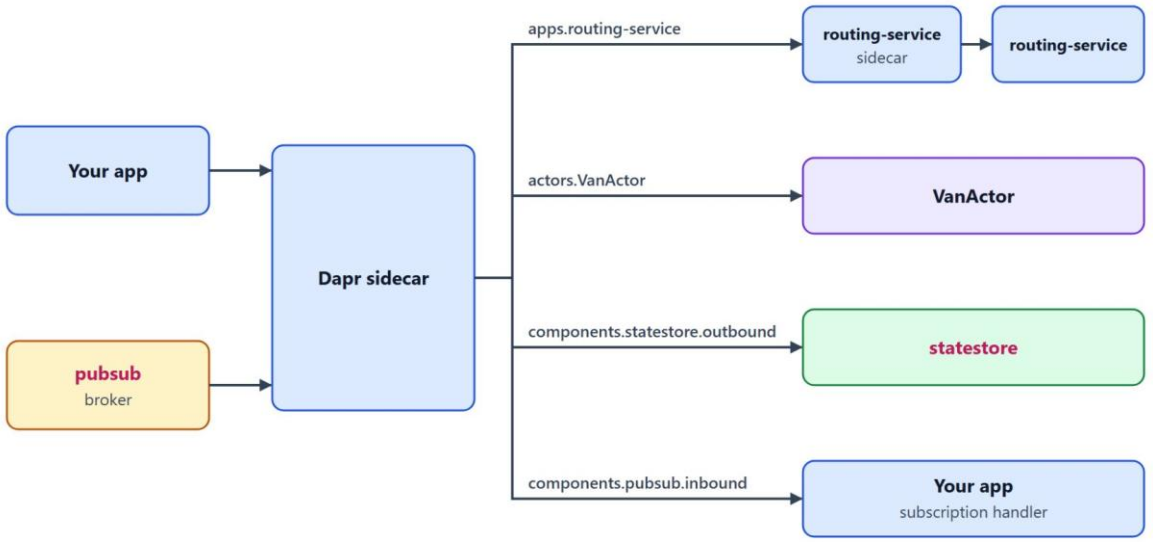
- `trip: consecutiveFailures >= 5` opens the breaker
- While open, the breaker rejects calls at once for its `timeout` of 60 s
- Half-open lets `maxRequests` (1) probe through, and a success closes it
- `interval` stays at 0 s, so nothing clears the count while the breaker is closed
- Dapr creates breakers per component and per app instance



Resiliency targets

Policies keyed by app ID, actor type or component name

- `apps` keys policies by app ID, `actors` by actor type, `components` by component name
- `outbound` protects the call to the store, `inbound` the delivery to your app
- Container Apps offers **Dapr component resiliency** as a preview feature
- `az containerapp env dapr-component resiliency create` attaches a policy to a component
- Service discovery resiliency does not apply to Dapr service invocation



06

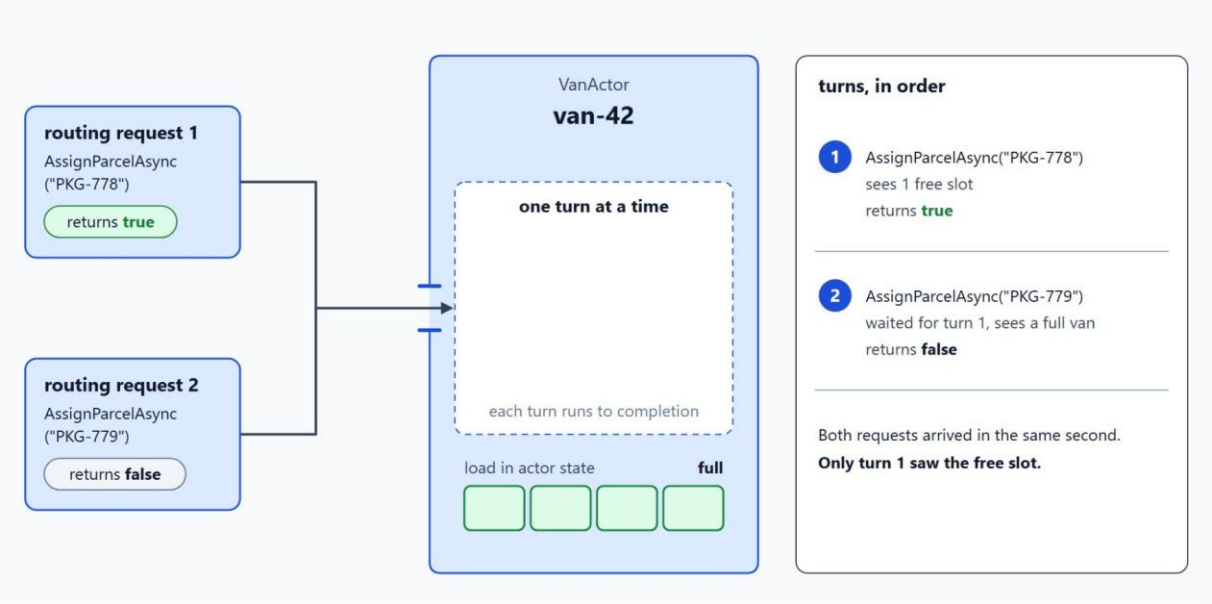
Dapr Actors: Virtual Actor Model

VanActor, turn-based calls, timers and reminders

Virtual actors and turn-based access

Each van is an actor that handles one call at a time

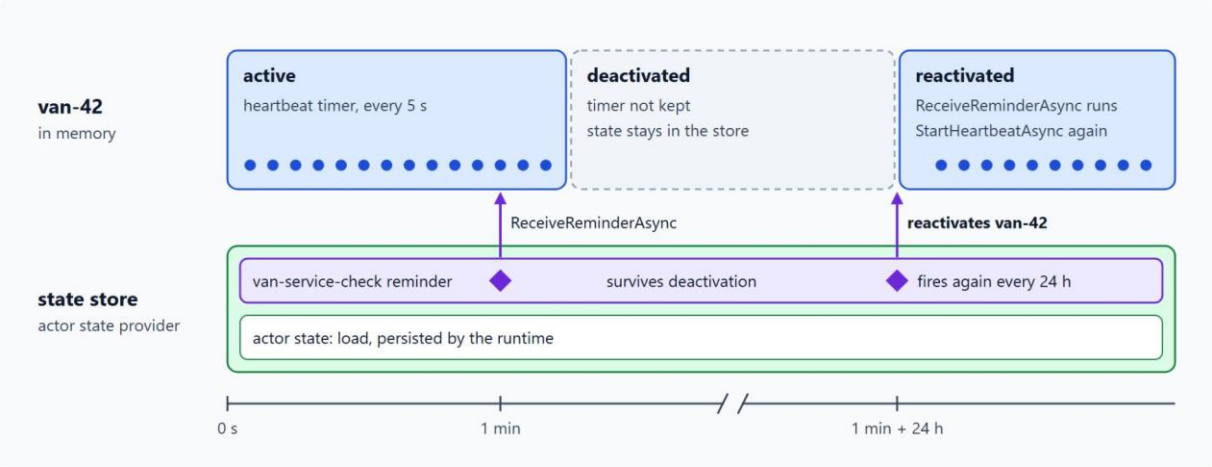
- The runtime activates an actor on the first request for its ID
- Idle actors are garbage-collected and come back with their saved state
- Calls run one at a time, so two assignments for `van-42` never share a free slot
- `IVanActor` derives from `IActor`, `VanActor` from `Actor`; callers use `ActorProxy.Create`
- All actor types share one **actor state store** that supports transactions



Timers and reminders

A heartbeat while the actor is active, a service check that survives deactivation

- `RegisterTimerAsync`: a heartbeat every 5 s, only while the actor is active
- `RegisterReminderAsync`: `van-service-check` every 24 h, persisted in the state store
- The runtime calls `ReceiveReminderAsync` for every reminder that fires
- Reminders cost more than timers, because the runtime writes them to storage
- Container Apps does not run the actor packages: use the AppHost or AKS



07

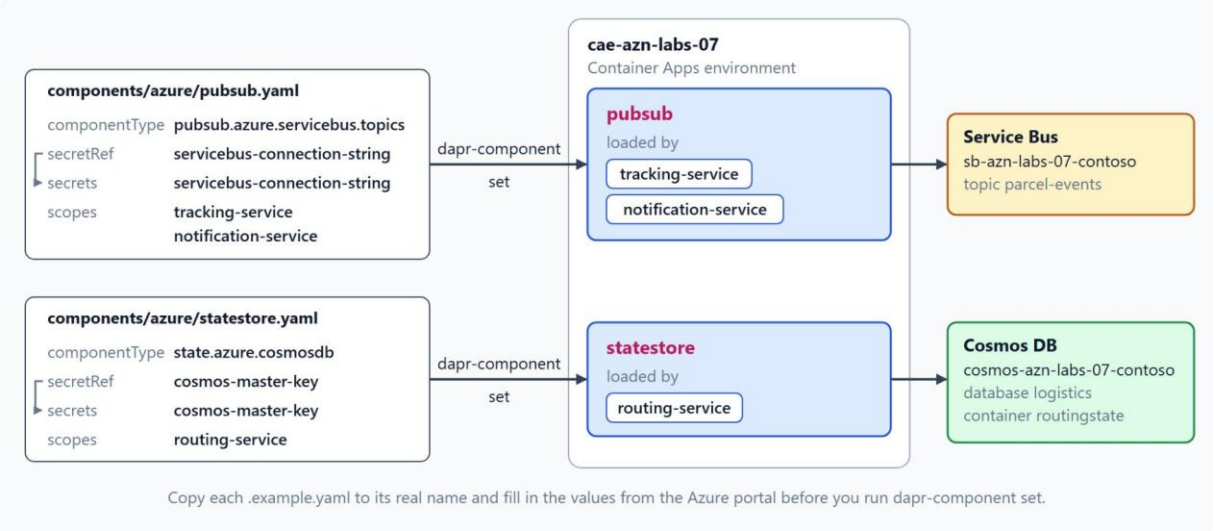
Managed Dapr: Container Apps vs. AKS

Component tiers, platform limits and the version window

Dapr on Container Apps

Components live on the environment and scopes pick the app IDs

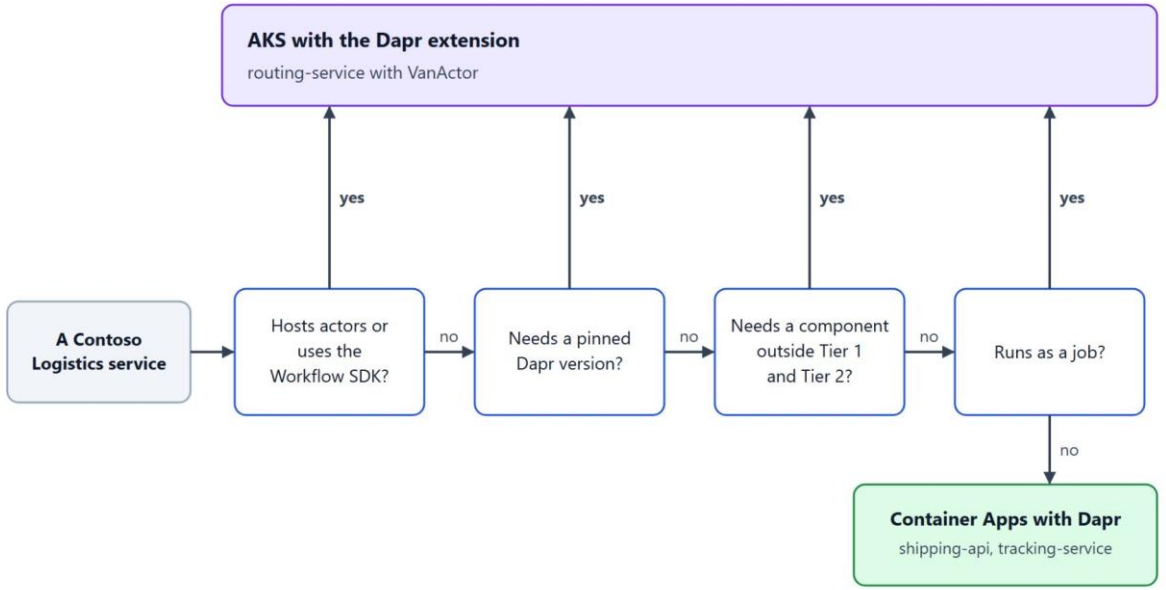
- `--enable-dapr` and `--dapr-app-id` turn Dapr on per app
- Components are defined once on the environment and scoped to app IDs
- Tier 1 and Tier 2 only: Cosmos DB state and Service Bus topics are **Tier 1**
- The platform picks and upgrades the Dapr version, with a suffix such as `-msft.1`
- No actor or workflow SDK packages, and no Dapr for jobs or Container Apps Express



Choosing Container Apps or AKS

Four questions decide where a service runs

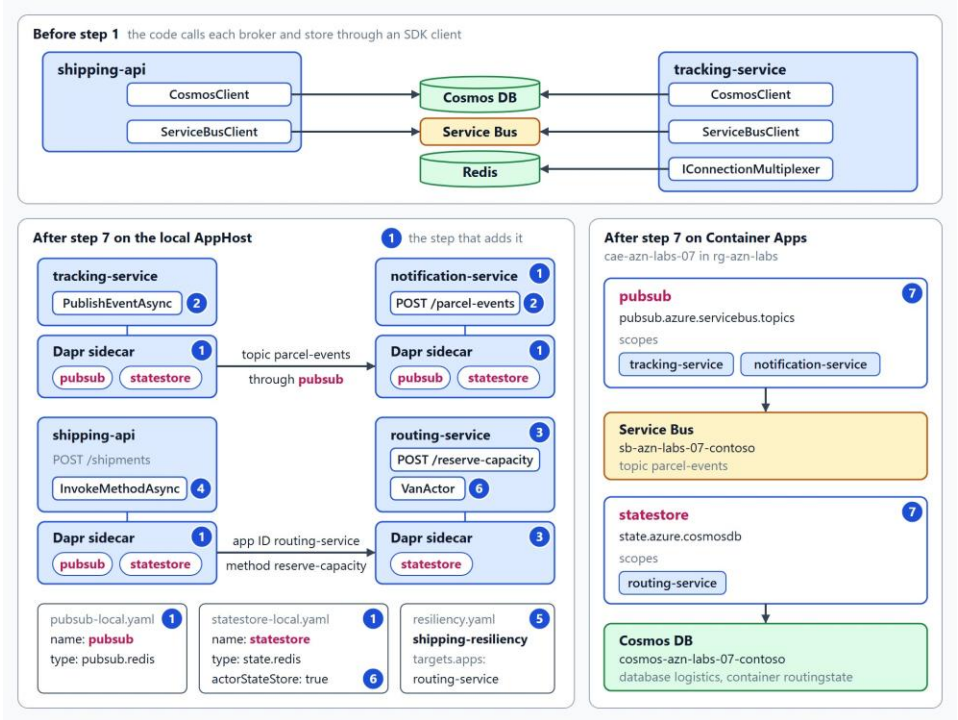
- AKS installs Dapr as the `Microsoft.Dapr` cluster extension
- The extension supports the current and the previous Dapr version
- Pin a version with `--version` and `--auto-upgrade-minor-version false`
- High availability and mTLS are on by default: three control-plane replicas
- Container Apps for stateless Tier 1 services, AKS for actors and pinned versions



What the lab builds

Scan, tracking, notification and routing flows on Dapr building blocks

- Sidecars with the Redis components `pubsub` and `statestore` (step 1)
- Scan events on `parcel-events`, with the tracking number as `SessionId` (step 2)
- routing-service: an assignment with an ETag check, called by app ID (steps 3 and 4)
- One resiliency spec and one `VanActor` per van (steps 5 and 6)
- The same component names on `cae-azn-labs-07`, backed by Service Bus and Cosmos DB (step 7)



Takeaways

What to carry into durable execution

- The code names a component, and the component file picks the Azure service
- Call services by app ID and leave the retries to the resiliency spec
- Write once-only transitions with an ETag and first-write-wins
- Give each van a `VanActor`, hosted on the AppHost or AKS
- Stateless Tier 1 services go to Container Apps, actors and pinned versions to AKS

